

Evulator

From Genomic Computation to Adaptive Machine Intelligence

Undergraduate edition

Chapter 1

Programs, genomes and the question of Evulator

A coordinated textbook series from Hermon AI

Internally reviewed development chapter · 6 September 2026

How to use this preview

This is the first chapter of a new undergraduate edition, not a completed textbook. Later chapter references point to the approved teaching plan. Earlier editions remain separately preserved.

Read pictures with their captions, try worked examples by hand, and attempt exercises before opening the answers. The Python section is optional. Linked glossary entries return to the section teaching each term; the index points to substantive definitions.

Figures are original editable vector artwork with Unicode TXT companions. Their captions state the simplifications. This preview contains no new laboratory result or claim of a research-model advantage.

Review so far is internal author-agent review using beginner and specialist perspectives. It is not independent expert certification, a learner study or an accessibility-conformance certification. The repository includes sources, tests, build instructions and a candid production report.

Contents

1	Programs, genomes and the question of Evolutor	1
1.1	Begin with a program that chooses	1
1.2	A boundary makes the rule precise	2
1.3	Optional: read the exact program	3
1.4	A biological reminder: stored DNA is not the whole story	4
1.5	What an analogy can and cannot carry	5
1.6	Evolutor begins as a research question	6
1.7	A route toward sharper questions	7
1.8	Practice: trace, compare, challenge	8
1.9	Answers and the question to keep	9
	Chapter 1 glossary	10
	Bibliography	13
	Index	14

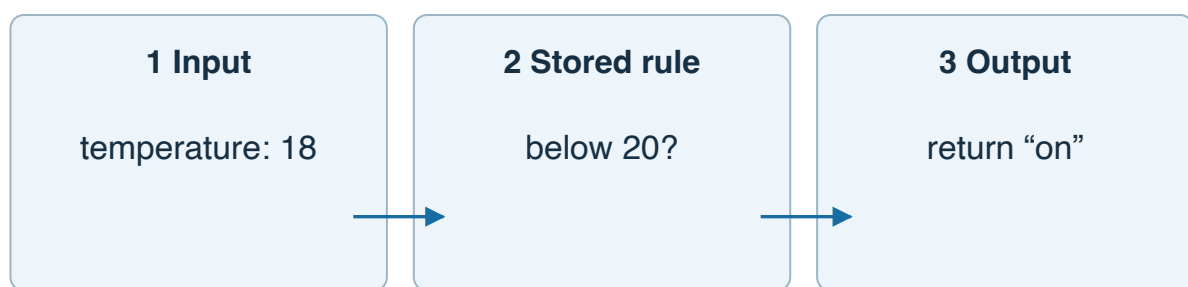
CHAPTER 1

Programs, genomes and the question of Evolutor

1.1 Begin with a program that chooses

Imagine a device that reads a temperature and produces the word “on” or “off.” Give it this instruction: if the temperature is below twenty, return “on”; otherwise return “off.” The same instruction can produce different answers. The difference comes from the temperature, not from rewriting the rule.

A **program** is instructions written in a form a computer can carry out. Its **input** is the starting information supplied to it. Its **output** is the resulting information we read, such as the word “on.” An **execution** means carrying out the program’s instructions.



Solid arrows: data used by the next step.

Output is a word, not a command sent to a real heater.

Figure 1.1: One execution of a fixed program. The temperature 18 makes “below 20?” true, so the program returns “on.” Arrows indicate use of information by the next step. No message is sent to a real heater.

A **rule** is an explicit instruction specifying an allowed operation. A **computation** carries out specified rules on represented information to obtain a result. These meanings agree with Book I. A fixed rule need not give the same result for every input.

KEY IDEA

A system can respond to its current situation without rewriting its instructions. Calling that response “biological” would not, by itself, identify a new computational mechanism.

By the end of this chapter, you should trace a small program, distinguish a biological gene from a proposed software unit and explain why an analogy needs a test. No knowledge of machine learning is required.

1.2 A boundary makes the rule precise

We must decide what “below twenty” means at exactly twenty. In ordinary arithmetic, twenty is not less than twenty. Our rule therefore returns “off.” Changing this choice would produce a different program.

A **threshold** is a chosen boundary used in making a decision. A **condition** is a statement checked as true or false. Here it asks whether the input is below the threshold. In symbols, $18 < 20$ says that eighteen is less than twenty; the pointed end of $<$ faces the smaller number.

A **trace** is a record of the steps and values in one execution. Figure 1.2 compares three short traces. The middle one checks the boundary instead of only easy cases far away from it.

INPUT	IS IT BELOW 20?	RETURNED WORD
18	yes	on
20	no: equal	off
22	no: above	off

The equality case is part of the specification, not an afterthought.

Figure 1.2: Three inputs, one unchanged rule. The highlighted middle row checks equality. Entries are exact outputs of the teaching rule, not measurements of a heating system. Labels preserve the distinction without color.

Worked example: temperature twenty. Step 1: read the input, twenty. Step 2: ask whether twenty is less than twenty. The answer is no. Step 3: follow “otherwise” and return “off.” Stop. No learning, memory or biological process is needed to explain this answer.

Context means the relevant current situation in which an operation is selected or carried out. Here the supplied temperature is the only context the program reads. Other systems may consider other information; that does not make every detail of the world an input to this program.

COMMON MISTAKE

A program is not necessarily rigid because its instructions are fixed. This program already chooses between results. A proposed new system must be compared with what ordinary programs can actually do.

1.3 Optional: read the exact program

Python is the programming language used here. A programming language supplies agreed ways to express executable instructions. Before the syntax, say the procedure aloud: receive a temperature; compare it with twenty; return “on” if it is lower; return “off” otherwise.

```
def heater(temperature):  
    if temperature < 20:  
        return "on"  
    return "off"
```

A **function** is a named, reusable rule in this program. The word `def` introduces the function named `heater`. Parentheses enclose `temperature`, the name standing for the input. A colon begins an indented block of instructions.

The word `if` checks the condition. The more deeply indented `return "on"` runs only when that condition is true. `return` gives back a result and finishes this execution. Otherwise, execution reaches the final line and returns `"off"`. Quotation marks mean that these are words, not names to look up.

Asking for `heater(20)` means apply the function to twenty. Its result is `"off"`. The table below is generated by running the actual source, which also supplies the printed listing.

Temperature	Below twenty?	Returned word
18	yes	on
20	no	off
22	no	off

To try it, open Python 3, type the printed definition, finish it with a blank line, then type `heater(18)`. Try twenty next. The official tutorial explains functions, conditions and return statements [1]. Running code is optional; predicting the outputs by hand is enough here.

WHY IT MATTERS

The hand trace, source code and generated table describe the same rule. If they disagree, something needs correcting. A program’s interesting name is no substitute for inspecting its behavior.

This example accepts ordinary numerical temperatures for a paper exercise. It does not validate sensors, handle missing readings or control equipment. It does not remember earlier inputs. Those are limits of this example, not evidence that all conventional programs lack those capabilities.

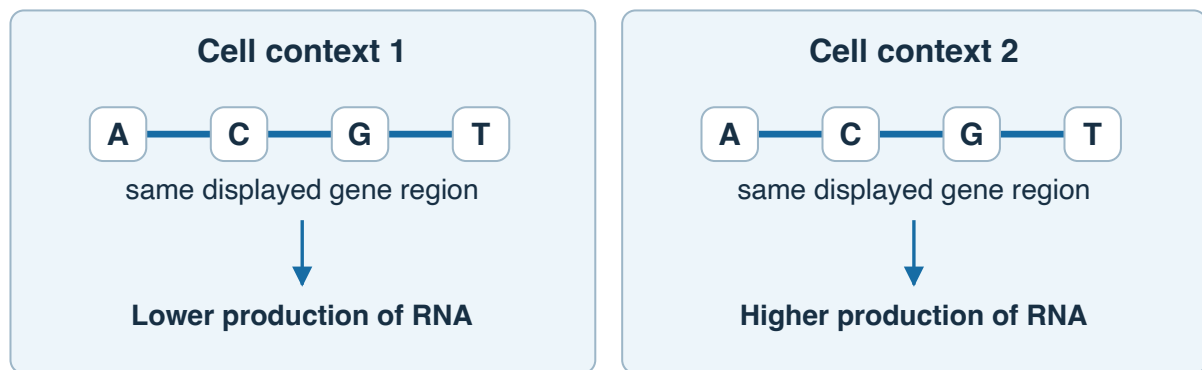
1.4 A biological reminder: stored DNA is not the whole story

Book I plans to teach programs in Chapter 3, DNA and genomes in Chapter 9, and regulation and evolution in Chapter 32. Those chapters are not yet in the undergraduate preview. This reminder supplies the biology needed here; it does not assume you have read unwritten material.

A **cell** is a basic living unit. A **molecule** is atoms joined by chemical bonds: small constituents of matter held together by chemical connections. **DNA** is deoxyribonucleic acid, a molecule that carries genetic information. A **genome** is an organism's complete set of DNA [3].

RNA means ribonucleic acid, a related molecule that can carry information from DNA and perform cellular roles. A **protein** is a molecule built from smaller units called amino acids; proteins do many structural and working jobs in cells [5, 6]. A **gene** is a DNA region whose information is used to make a functional RNA or, through RNA, a protein [2]. These are working descriptions, not a complete chemistry lesson.

Gene expression is the use of a gene's information to produce RNA or protein. Here, **gene regulation** is control over when, where or how much that expression occurs. Expression can change without changing the DNA sequence, its written order [4].



Arrows: RNA production using cellular machinery, not DNA movement.

Conceptual comparison; not measured rates or a complete cell.

Figure 1.3: The same displayed DNA region can be used differently in different cellular contexts. Arrows mean RNA production using cellular machinery. Lower and higher are conceptual, not measured rates. The picture does not say DNA acts alone or every cell always has identical DNA [4].

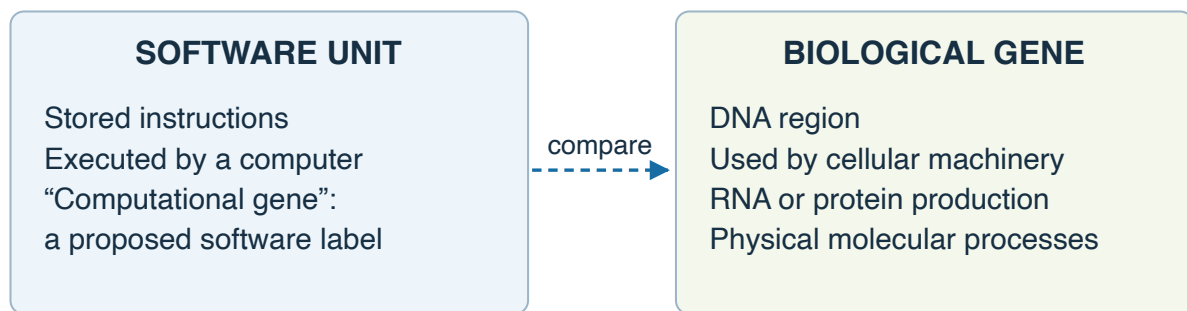
A genome does not simply “run itself.” Molecular machinery and surroundings matter. This picture suppresses the machinery to compare one feature: the shown DNA can remain the same while its use differs. It is not a complete cell mechanism.

1.5 What an analogy can and cannot carry

The program and the biological example share a limited pattern: stored possibilities are used differently in different situations. Is that resemblance useful? Perhaps—but it is not yet a mechanism.

An **analogy** is a comparison highlighting a limited resemblance between different things. It can suggest a question while failing as an explanation. A map resembles the arrangement of streets but cannot carry a passenger; resemblance does not transfer every ability.

A **computational gene** is a proposed named software unit, not a DNA molecule. This provisional label leaves work for later chapters: what does the unit store, how is it selected, and what does executing it mean? The label alone does not identify a working architecture.



Resemblance: stored possibilities used in a context.

Not identity: names do not transfer biological capabilities.

Figure 1.4: Keep mechanisms in separate lanes. The dashed arrow means “compare,” not molecular conversion, software execution or equivalence. A biological gene and a proposed computational gene do not become the same kind of object because a name is shared.

Worked comparison. First, identify the program’s mechanism: a numerical comparison selects a returned word. Second, identify the biological claim shown: gene expression can vary with context. Third, name the resemblance: context affects an outcome. Finally, name the gap: the resemblance specifies no new software mechanism and establishes no advantage over ordinary conditional instructions.

COMMON MISTAKE

“Both respond to context” is too weak to establish novelty. The heater program already does that. A proposal must specify what it does differently and why that difference might matter for a task.

Biological language sometimes names established biology and sometimes labels a proposed software abstraction. That distinction must stay visible in prose, pictures and claims. We will not draw software as a molecule merely to make it look biological.

1.6 Evolver begins as a research question

A **research program** is a coordinated set of questions, proposed explanations and tests. Evolver is introduced here in that sense. It asks whether selected principles of genomic organization can inspire useful computational mechanisms. It is not introduced as a demonstrated solution to general intelligence.

A **hypothesis** is a claim precise enough to be tested and potentially contradicted. A **baseline** is an ordinary comparison method against which a proposal is assessed. Here, **evidence** is an observation or reason bearing on a stated claim. Calling a design genomic supplies none of these automatically.

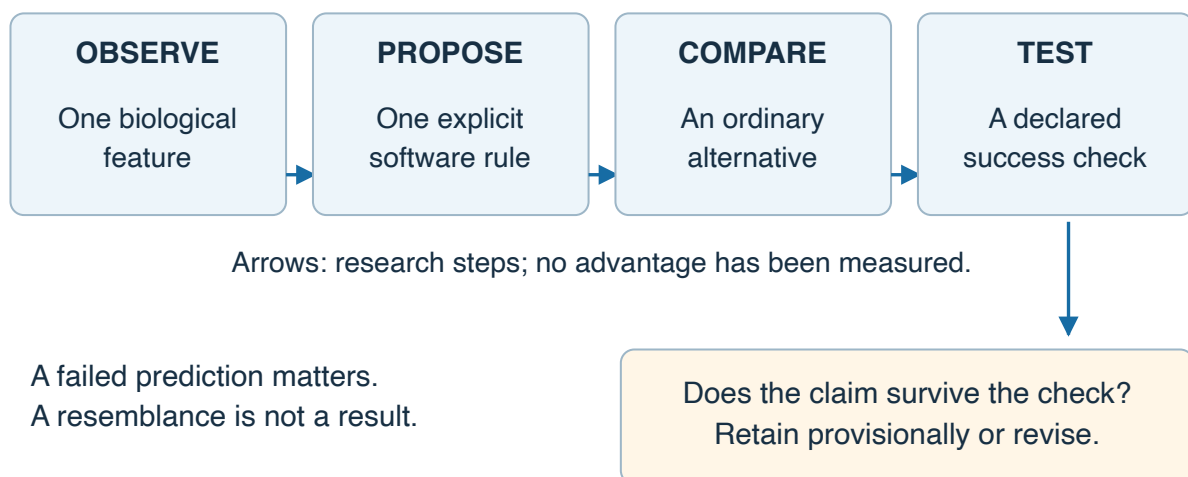


Figure 1.5: From inspiration to a claim that can fail. Solid arrows organize research work. Comparison and testing are obligations, not boxes certifying success. A claim surviving one check is retained provisionally, not proved universally.

Consider: “Choosing only needed operations will reduce work on this task.” This proposal needs a definition of needed, a way to count work and an ordinary alternative. The alternative must be allowed to make ordinary choices too. Otherwise the test could reward a new label for a capability artificially denied to its competitor.

Before a test, state what must stay comparable: the task, permitted inputs and required correctness. Then define the resource counted, such as executed operations. A lower count is not automatically lower elapsed time; fewer operations would not help if the answer became unacceptable. This is a design exercise, not a reported result.

RESEARCH FRONTIER

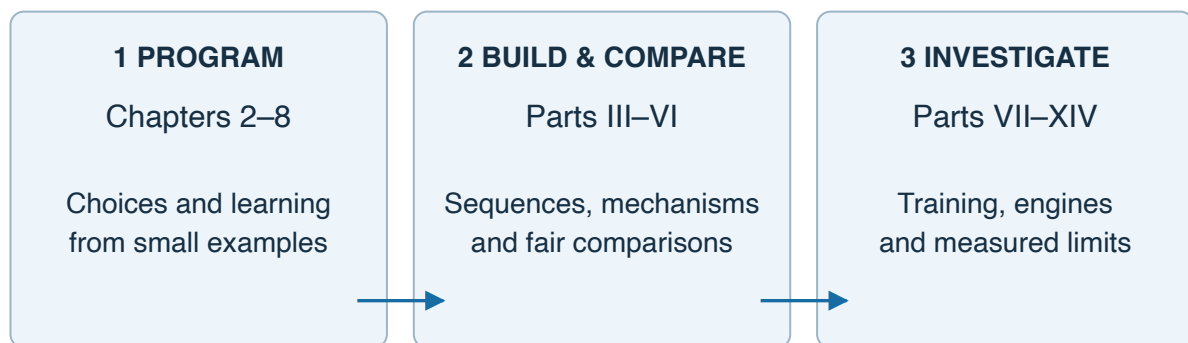
This chapter establishes no advantage from genomic organization. It reports no trained model, benchmark win or general-intelligence result. The question is whether a specified mechanism adds useful capability beyond suitable conventional alternatives.

An unfavorable comparison matters: the resemblance may not be the useful part, or the task may not benefit from it. A research program needs room for that answer.

1.7 A route toward sharper questions

A comparison is only as fair as our understanding of the alternatives. The next chapter, *Programs that choose and remember*, develops choices and stored state—information retained between steps. It will provide a local programming bridge while corresponding Book I chapters remain planned.

A **representation** is a chosen way to express information through distinguishable forms. The term **information** here means what a chosen description tells us about the possibilities relevant to a task. The words “on” and “off” distinguish two possible returned answers. These meanings match Book I; no mathematical measure of information is assumed.



Arrows: teaching order. These later chapters are still planned.

Figure 1.6: The planned learning route. Learning here previews adjustment from examples, beginning in Chapter 3. The diagram supplies no model mechanism or extra prerequisite. Arrows mean teaching order. Only Chapter 1 is produced in this undergraduate preview.

Chapter 3 introduces systems that adjust numerical choices from examples. Detailed mechanisms follow their mathematical and programming foundations. We do not need them to ask today’s question: what is proposed, what already exists, and what counts as evidence?

Established: the printed program follows its rule; gene expression can differ with context. **Simplified:** a four-line program and a two-context biological picture omit most engineering and biology. **Analogy:** context affects outcomes in both examples. **Hypothesis:** a biological principle might inspire a useful new computational mechanism. These are different kinds of statement.

CHECK YOUR UNDERSTANDING

Explain why the word genome cannot guarantee useful software behavior. Propose one observation that would make a particular software claim more credible. If no imaginable check could challenge the claim, sharpen it.

The point is not to lower ambition but to make it examinable. A useful later chapter must turn a question into a mechanism and expose that mechanism to a comparison it might lose.

1.8 Practice: trace, compare, challenge

Begin with concrete cases. The last questions ask for reasons, not optimistic predictions about the research program.

1. **Recognize.** Name the input, stored rule and returned output when the temperature is eighteen.
2. **Trace.** What happens at twenty? Why is “on” incorrect for the rule as written?
3. **Change one instruction.** If “below twenty” becomes “twenty or below,” which printed case changes? What stayed fixed?
4. **Optional code.** Predict heater(19). Which line supplies the result? Why is the final line not reached?
5. **Biological recognition.** A picture shows the same DNA region but different RNA production in two contexts. Does this show a DNA change? Does it show that DNA needs no cellular machinery?
6. **Limit an analogy.** Give one resemblance and one difference between the program and the biological example. Do not merely call both smart.
7. **Make a claim testable.** Replace “our genomic software is better” with a small claim naming a task, an ordinary alternative and a check. What result would count against it?
8. **Exit explanation.** A proposal renames every function a gene. Its author says it must now be more adaptive. What evidence is missing, and why is the name insufficient?

BEFORE YOU CHECK

For biology, ask which physical process is described. For software, ask which instructions are specified. For research, ask which comparison could challenge the claim.

You are ready for the next step when you can perform the trace and keep these statements separate. You need no verdict on the whole Evulator project, only a reliable way to inspect its claims.

1.9 Answers and the question to keep

1. Input: eighteen. Rule: return “on” below twenty, “off” otherwise. Output: “on.” The threshold remains twenty.
2. Twenty is not less than twenty. The condition is false and the other result is “off.” Returning “on” would follow a different boundary rule.
3. Only equality changes, from “off” to “on.” Inputs stay fixed; the comparison changes. Eighteen still qualifies and twenty-two still does not.
4. “on”. Nineteen is below twenty, so the indented return runs and finishes the function. The final return is not reached.
5. Neither. Expression may change while the shown DNA remains fixed. Cellular machinery is explicitly required, though not drawn.
6. Resemblance: context affects an outcome. Difference: software evaluates a numerical condition; biological expression involves molecules and cellular processes. The program does not prove that a gene is an if-statement.
7. One claim: “On these specified inputs, the proposed selector executes fewer operations than this ordinary conditional program while returning the same answers.” A wrong answer or no reduction counts against it. A real test still needs a precise counting convention and a credible alternative.
8. Renaming changes neither instructions nor measured behavior. Specify a changed mechanism, a meaning of adaptive, a task and a fair comparison. Then report observations, including failures. None follows from a naming change.

Return to the opening. An unchanged program already chooses according to an input. Biology supplies richer phenomena, but richness is not itself a software specification. Can we select a principle, specify a computational mechanism and show that it helps under a fair comparison?

That question introduces Evolutor. Later chapters must earn stronger answers. This one gives the reader tools to ask for them.

Chapter 1 glossary

analogy

is a comparison highlighting a limited resemblance between different things. First taught in Section 1.5; see also [computational gene](#).

baseline

is an ordinary comparison method against which a proposal is assessed. First taught in Section 1.6; see also [hypothesis](#).

cell

is a basic living unit. First taught in Section 1.4; see also [genome](#).

computation

carries out specified rules on represented information to obtain a result. First taught in Section 1.1; see also [rule](#).

computational gene

is a proposed named software unit, not a DNA molecule. First taught in Section 1.5; see also [analogy](#).

condition

is a statement checked as true or false. First taught in Section 1.2; see also [threshold](#).

context

means the relevant current situation in which an operation is selected or carried out. First taught in Section 1.2; see also [condition](#).

DNA

is deoxyribonucleic acid, a molecule that carries genetic information. First taught in Section 1.4; see also [molecule](#).

evidence

is an observation or reason bearing on a stated claim. First taught in Section 1.6; see also [hypothesis](#).

execution

means carrying out the program's instructions. First taught in Section 1.1; see also [trace](#).

function

is a named, reusable rule in this program. First taught in Section 1.3; see also [program](#).

gene

is a DNA region whose information is used to make a functional RNA or, through RNA, a protein. First taught in Section 1.4; see also [gene expression](#).

gene expression

is the use of a gene's information to produce RNA or protein. First taught in Section 1.4; see also [gene regulation](#).

gene regulation

is control over when, where or how much that expression occurs. First taught in Section 1.4; see also [gene expression](#).

genome

is an organism's complete set of DNA. First taught in Section 1.4; see also [gene](#).

hypothesis

is a claim precise enough to be tested and potentially contradicted. First taught in Section 1.6; see also [evidence](#).

information

here means what a chosen description tells us about the possibilities relevant to a task. First taught in Section 1.7; see also [representation](#).

input

is the starting information supplied to it. First taught in Section 1.1; see also [output](#).

molecule

is atoms joined by chemical bonds: small constituents of matter held together by chemical connections. First taught in Section 1.4; see also [DNA](#).

output

is the resulting information we read, such as the word "on.". First taught in Section 1.1; see also [input](#).

program

is instructions written in a form a computer can carry out. First taught in Section 1.1; see also [function](#).

protein

is a molecule built from smaller units called amino acids; proteins do many structural and working jobs in cells. First taught in Section 1.4; see also [RNA](#).

Python

is the programming language used here. First taught in Section 1.3; see also [program](#).

representation

is a chosen way to express information through distinguishable forms. First taught in Section 1.7; see also [information](#).

research program

is a coordinated set of questions, proposed explanations and tests. First taught in Section 1.6; see also [hypothesis](#).

RNA

means ribonucleic acid, a related molecule that can carry information from DNA and perform cellular roles. First taught in Section 1.4; see also [protein](#).

rule

is an explicit instruction specifying an allowed operation. First taught in Section 1.1; see also [computation](#).

threshold

is a chosen boundary used in making a decision. First taught in Section 1.2; see also [condition](#).

trace

is a record of the steps and values in one execution. First taught in Section 1.2; see also [execution](#).

Bibliography

- [1] Python Software Foundation. *The Python Tutorial: More Control Flow Tools*. Sections on if statements, functions and return.
<https://docs.python.org/3/tutorial/controlflow.html>. Accessed 6 September 2026.
Examples tested locally with Python 3.13.6.
- [2] National Human Genome Research Institute. *Gene*.
<https://www.genome.gov/genetics-glossary/Gene>. Accessed 6 September 2026.
- [3] National Human Genome Research Institute. *Genome*.
<https://www.genome.gov/genetics-glossary/Genome>. Accessed 6 September 2026.
- [4] Bruce Alberts, Alexander Johnson, Julian Lewis, Martin Raff, Keith Roberts and Peter Walter (2002). An Overview of Gene Control. In *Molecular Biology of the Cell*, fourth edition. <https://www.ncbi.nlm.nih.gov/books/NBK26885/>. Used for context-dependent expression and the distinction between DNA and its use. Historical gene-count estimates are not used.
- [5] National Human Genome Research Institute. *Ribonucleic Acid (RNA)*.
<https://www.genome.gov/genetics-glossary/Ribonucleic-Acid-RNA>. Accessed 6 September 2026.
- [6] National Human Genome Research Institute. *Protein*.
<https://www.genome.gov/genetics-glossary/Protein>. Accessed 6 September 2026.

Index

analogy, 5

baseline, 6

cell, 4

computation, 1

computational gene, 5

condition, 2

context, 2

DNA, 4

evidence, 6

execution, 1

function, 3

gene, 4

gene expression, 4

gene regulation, 4

genome, 4

hypothesis, 6

information, 7

input, 1

molecule, 4

output, 1

program, 1

protein, 4

Python, 3

representation, 7

research program, 6

RNA, 4

rule, 1

threshold, 2

trace, 2